

Hi-Tec Shopify Automation

A Brief

Portfolio explainer, built from the repository source

1 What it is

Three small Python programs, written solo as freelance work for Hi-Tec Bearings in 2023. The client sells bearings through a Shopify shop with thousands of product variants. The stock figures they trust live in a spreadsheet exported from their warehouse system; what the shop tells customers lives in a different spreadsheet exported from Shopify. Reconciling the two by hand took hours per cycle. These tools do it automatically.

The defining constraint: no API

Nothing here talks to Shopify over the network. No API client, no access token, no shop domain. A human exports a spreadsheet from the Shopify admin, runs a tool, reviews the output, and re-imports it. Everything in the design follows from that.

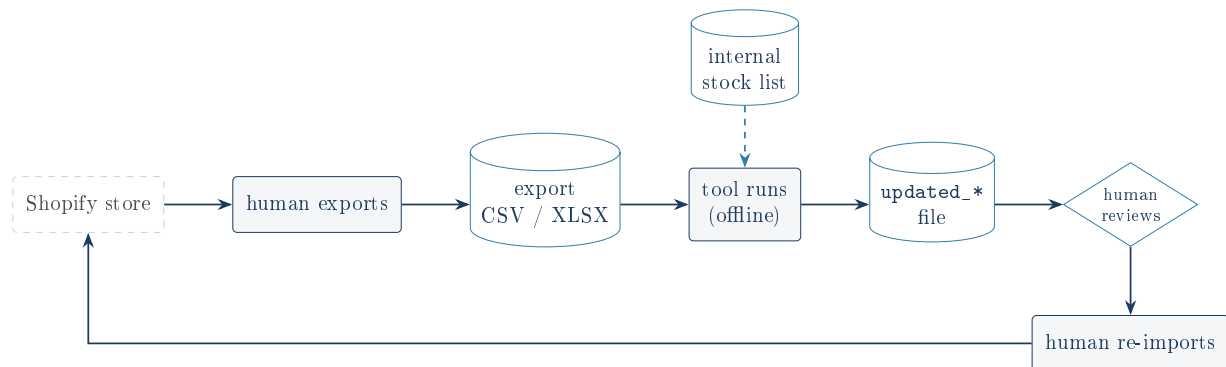


Figure 1: The round trip. The tools are one box; the human is four, and one of them is a review gate no catalogue change can skip.

The constraint costs and pays. It pays because a wrong result is a wrong spreadsheet rather than a wrong live storefront. It costs because without the API there are no stable product IDs to join on, only whatever text the export happens to contain. That is the whole engineering problem.

2 The three tools

Tool	Does	Notes
<code>quantity_pricing_updater</code>	Merges warehouse quantity and average cost into a Shopify export	CLI. 69 lines. Verified working.
<code>dimensions_extractor</code>	Pulls bore / outside diameter / width out of HTML descriptions into columns	CLI. 83 lines. Verified working , with a row-dropping defect.
<code>stock_updater_gui</code>	tkinter app: both of the above, plus variant detection, behind three buttons	354 lines. The client's deliverable. Carries a serious bug.

Both CLI tools were shipped to the client as standalone Windows executables via PyInstaller, so no Python install was needed.

3 The central idea: matching without a key

Two tables, no shared column

The stock list has no Shopify ID. The Shopify export has no stock item number. The only thing in both is the bearing designation as text, e.g. 6205. And it is not unique: 6205 appears once per brand, and only the HI-TEC row is the one to trust. **Filter by brand first, then join.** That single decision is what the project turns on.

The join key then depends on the product's shape. Shopify gives every product at least one variant, labelling a variant-less product's row `Option1 Value = Default Title`. So:

- `Default Title` rows: join on `Title`.
- Variant rows: join on `Option1 Value`, because Shopify writes the `Title` only on the first row of each product group and leaves it empty on the rest.

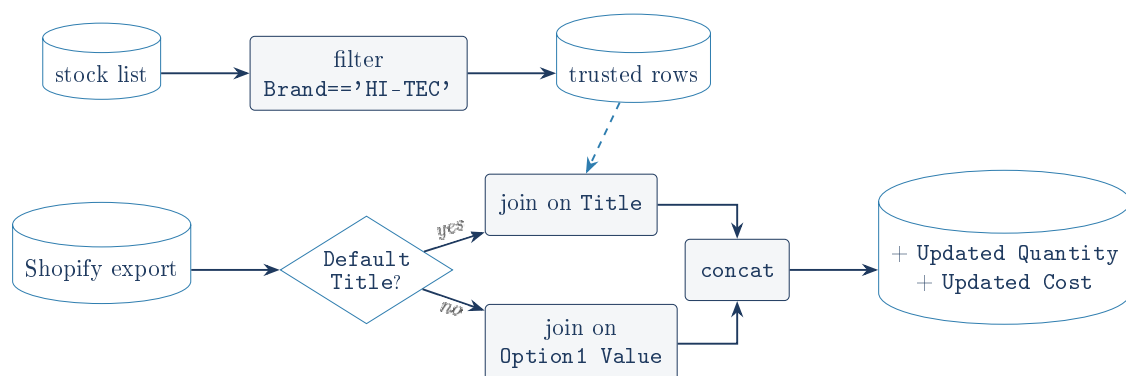


Figure 2: Brand filter, then a split on the `Default Title` sentinel to choose the join key.

Verified by running it. The shipped sample deliberately includes trap rows: 6205 and 6304 2RS each have a competing `OTHER` brand row with different figures. The tool takes the HI-TEC figures for both, and matches both variant rows through `Option1 Value` including the one with a blank `Title`. The README's main claim about this tool is true.

4 The second idea: data locked in HTML

Bearing dimensions existed only as an HTML table inside each product’s description, so the storefront could not filter on them. The extractor lowercases the description, strips every tag with the regex `<.*?>`, splits on newlines, drops blanks, and then relies on one assumption: **a label line is immediately followed by its value line**, so `['bore diameter', '25']` are adjacent. Find the label, take the next line.

It works, and it recovered all three dimensions from every sample row that had a description. It is also brittle by construction: it rides on the store’s exact description template and would silently miss dimensions on any other layout. The README says this plainly and is right to.

One detail worth knowing: a **total width** fallback exists for products that label the field differently. That line is domain knowledge learned from the real catalogue, not something anyone would write from first principles.

5 Findings

The GUI’s quantity matcher matches nothing on the shipped sample

The GUI is not a wrapper around the two CLI tools; it is a copy of them, and the copies have drifted. The GUI has one line its CLI sibling lacks:

```
1 stock_list['Item'] = stock_list['Item'].apply(convert_int_float)
```

It is meant to normalise types. It does two damaging things. `'6205'` becomes the float `6205.0`, which never compares equal to the string `'6205'` in the Shopify `Title` column. And `'22210 K'` cannot parse as a float, so the function falls through to a bare `pass`, returns `None`, and pandas stores `nan`: every alphanumeric designation, meaning every variant, is destroyed.

Running the GUI’s exact matching logic against the sample: **0 of 4** default-title matches, **0 of 2** variant matches. The CLI matches all six. The GUI would write zeros for every product, report “Quantity and Pricing has been updated!” in green, and save the file.

Scope, stated precisely. What is verified is the failure on the sample in this repository. Whether it failed on the client’s real 2023 files is **unresolvable from this repo**, because it depends on the dtypes pandas inferred from data that is no longer here. If the real columns were purely numeric the line was harmless; if either contained one alphanumeric designation, and the sample says the real catalogue was full of them, the match collapses exactly as it does here. The second is far more likely.

The honest position, and a strong one: the CLI implementation is correct and verifiably so; the GUI copy carries a well-understood dtype bug; and that bug is the direct, predictable consequence of the code duplication the README already names as the project’s main design regret. The fix is two lines.

The “11,000-line Shopify export” is not in this repository

The README says, correctly and in the past tense, that the original repo held the client’s real data (about 1,600 stock rows and an 11,000-line Shopify export) and that it was removed. Checked: what is actually here is a fabricated sample of **8 stock rows** and **6 product rows** (117 physical lines, since the HTML descriptions span many lines each).

So those figures rest entirely on the owner’s recollection. Git history confirms the real data was **never committed**, so it cannot be recovered from history either. Good for confidentiality; it also means the numbers are unverifiable by anyone without the original client files. Not an accusation that they are wrong, a note that they are the one class of claim here that cannot be defended by pointing at code. The sample demonstrates correctness, at six rows. It does

not demonstrate scale.

The dimensions extractor deletes rows from its own output

`df = df.dropna(subset=['Body (HTML)'])` removes every row without a description. Since Shopify writes product-level fields only on the first row of a product group, that is most variant rows. Verified: 6 rows in, 5 rows out; the 22210 E variant vanished. The tool's stated contract is "adds three columns"; its behaviour is "adds three columns and silently drops rows". The one-line fix is to filter the loop's iteration source rather than the frame.

Smaller, but real

- `save_data` uses a full `read_excel` parse as an existence check, and catches only `FileNotFoundError`. The previous output being open in Excel raises `PermissionError` instead, which is uncaught. For a client who lives in Excel, that is the likeliest real-world failure.
- `open_file`'s retry loop cannot retry: both `if` branches return. Cancelling the dialog yields an empty path and an unhandled exception.
- Nothing writes to the status widget on failure, only on success.
- Matching is $O(n \times m)$: the stock list is re-filtered and re-listed on every Shopify row. One `merge` would be linear and shorter.
- The `Seal Type` mapping is three-quarters a no-op: only `2rs` is translated, the rest map to themselves.
- An unmatched product gets `Updated Quantity = 0`, indistinguishable from genuinely out of stock, in a column that feeds a live storefront.
- Debug prints throughout, in the version whose likely packaging (`console=False`, **inferred**: the GUI's `.spec` is absent) discards stdout entirely.
- No tests, though the sample data is already an adversarial fixture.

Security: clean, by construction

No credentials exist in this project because none can: no API client, no tokens, no shop domain, no network calls. `.env.example` declares that nothing is needed. A scan of the tree found nothing, and git history was checked: the client's real data was never committed, so there is nothing to scrub. The sample is fabricated and the README says so.

6 What is genuinely good

- **The brand-filter insight.** Seeing that the join is ambiguous and that `Brand` is the disambiguator is the thinking the project turns on. It removed a class of wrong match no fuzzy title matching would have fixed.
- **Reading Shopify's export conventions correctly**, including the `Default Title` sentinel and the empty `Title` on variant rows.
- **Domain knowledge that came from the data**, not from assumptions: the `total width` fallback, and a `series-2xxxx` rule in the variant detector that deliberately overrides the cage lookup table.
- **Delivery treated as a real requirement.** The GUI, the executables, the file pickers, and

a non-overwrite save that bumps a counter exist because the user was a person rather than a terminal.

- **An honest scrub.** Replacing client data with a fabricated fixture that keeps the same column layout, the same HTML structure, and deliberate brand collisions, so the tools still run and can be verified, is better than most portfolio scrubs manage.

7 What was verified, and what could not be

CLI merge picks HI-TEC rows, matches variants	Verified by running it
CLI extractor recovers all three dimensions	Verified by running it
Extractor drops description-less rows	Verified by running it
GUI matcher matches nothing on the sample	Verified by running its logic
GUI window matches the shipped screenshot	Verified
No credentials in tree or history	Verified
GUI's real .spec used console=False	Inferred, not in repo
Shopify import ignores absent rows	Inferred, not tested
GUI bug's effect on the real 2023 data	Unresolvable from this repo
1,600 stock rows / 11,000-line export	Unverifiable, never committed
End-to-end live round trip (2023)	Not re-verified, as README says

The fair summary

Three small tools that solved a real and genuinely awkward problem under a constraint that forced text matching on data with no shared key. The central idea is sound and verifiably works in the CLI tools. The delivered GUI is a copy-paste fork that carries a serious silent bug, and the absence of both tests and a shared module is exactly why that bug could survive unnoticed. The author has already named most of this project's weaknesses accurately in his own README; the GUI matcher is the one he had not found, and it is the most interesting thing here.